Source Code :-

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
Dropout
from tensorflow.keras.optimizers import Adam, SGD, RMSprop
from tensorflow.keras.regularizers import l2

(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()

# Normalize pixel values
x_train = x_train / 255.0
x_test = x_test / 255.0

# Reshape data for CNN
x_train = x_train.reshape(-1,28,28,1)
x_test = x_test.reshape(-1,28,28,1)

print("Training Data Shape:", x_train.shape)
print("Testing Data Shape:", x_test.shape)
Training Data Shape: (60000, 28, 28, 1)
Testing Data Shape: (10000, 28, 28, 1)
def create_model(filter_size=3, regularization=False, optimizer='adam'):

    if regularization:
        reg = l2(0.001)
    else:
        reg = None

    model = Sequential()

    model.add(Conv2D(32,(filter_size,filter_size),
                     activation='relu',
                     kernel_regularizer=reg,
                     input_shape=(28,28,1)))
```

```
    model.add(MaxPooling2D((2,2)))

    model.add(Conv2D(64,(filter_size,filter_size),activation='relu'))

    model.add(MaxPooling2D((2,2)))

    model.add(Flatten())

    model.add(Dense(128,activation='relu'))

    model.add(Dropout(0.3))

    model.add(Dense(10,activation='softmax'))

    if optimizer == 'adam':
        opt = Adam()
    elif optimizer == 'sgd':
        opt = SGD()
    else:
        opt = RMSprop()

    model.compile(
        optimizer=opt,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
model = create_model()

history = model.fit(
        x_train,
        y_train,
        epochs=10,
        batch_size=64,
        validation_split=0.2,
        verbose=1
)
test_loss, test_acc = model.evaluate(x_test,y_test)
```

```
print("\nTest Accuracy:",test_acc)
print("Test Loss:",test_loss)
```

Output :-

```
Test Accuracy: 0.9089999794960022
Test Loss: 0.254584938287735
```

```
epoch_table = pd.DataFrame(history.history)
print("\nEpoch-wise Performance\n")
print(epoch_table)
```

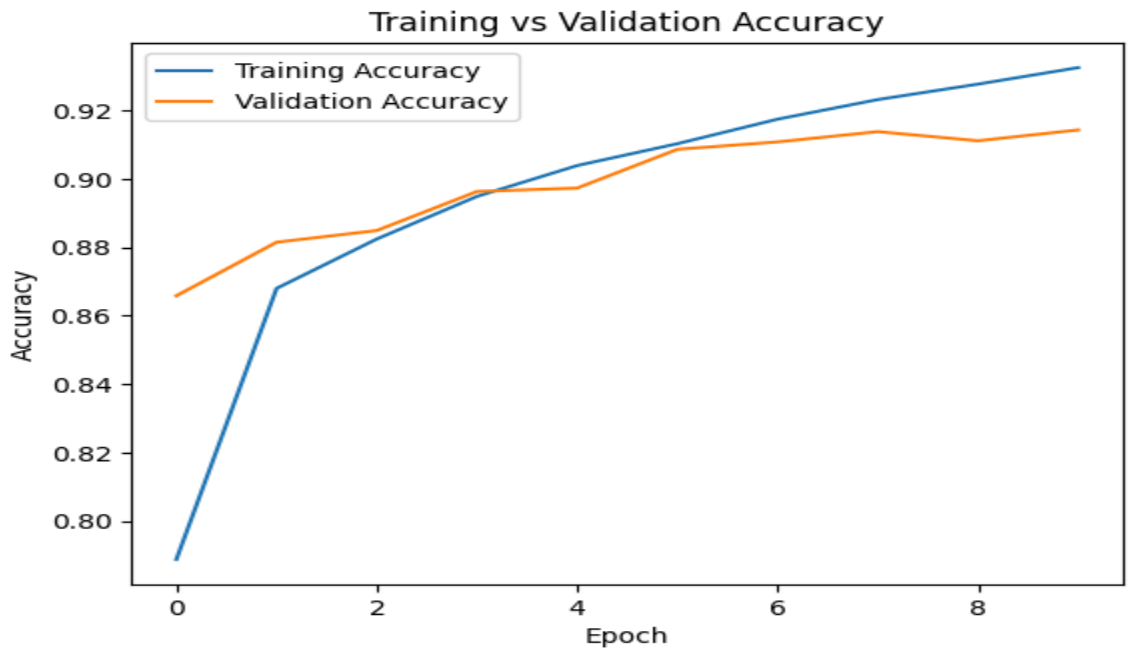
```
Epoch-wise Performance
```

Output :-

```
   accuracy      loss  val_accuracy  val_loss
0  0.788750  0.571954      0.865833  0.371731
1  0.868000  0.366708      0.881500  0.325045
2  0.882438  0.319284      0.884917  0.307075
3  0.894896  0.287117      0.896333  0.282052
4  0.903917  0.262578      0.897333  0.273392
5  0.910312  0.241184      0.908667  0.248109
6  0.917479  0.224634      0.910833  0.240539
7  0.923208  0.206786      0.913833  0.237461
8  0.927729  0.192864      0.911167  0.250773
9  0.932562  0.180979      0.914333  0.240931
```

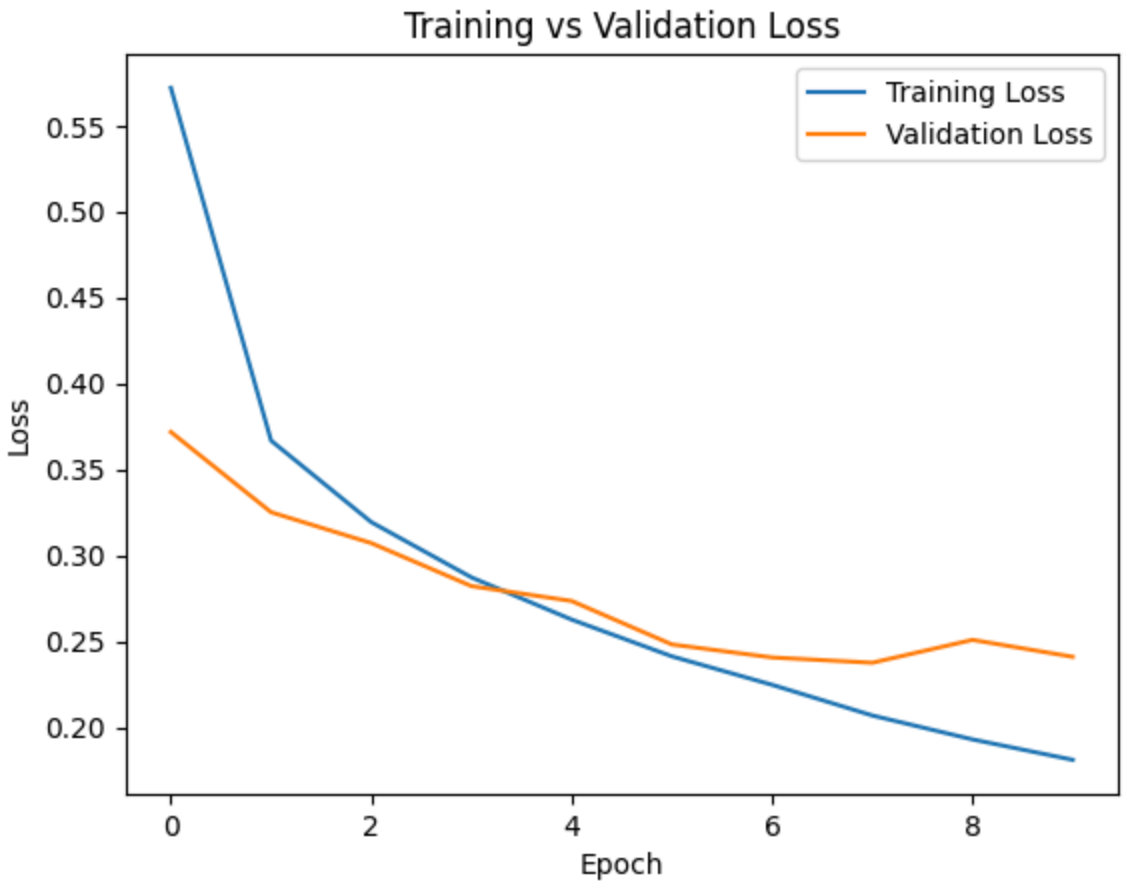
```
plt.figure()

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title("Training vs Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Training Accuracy","Validation Accuracy"])
plt.show()
```

Output :-




```
plt.figure()

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title("Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Training Loss","Validation Loss"])

plt.show()
```

Output :-

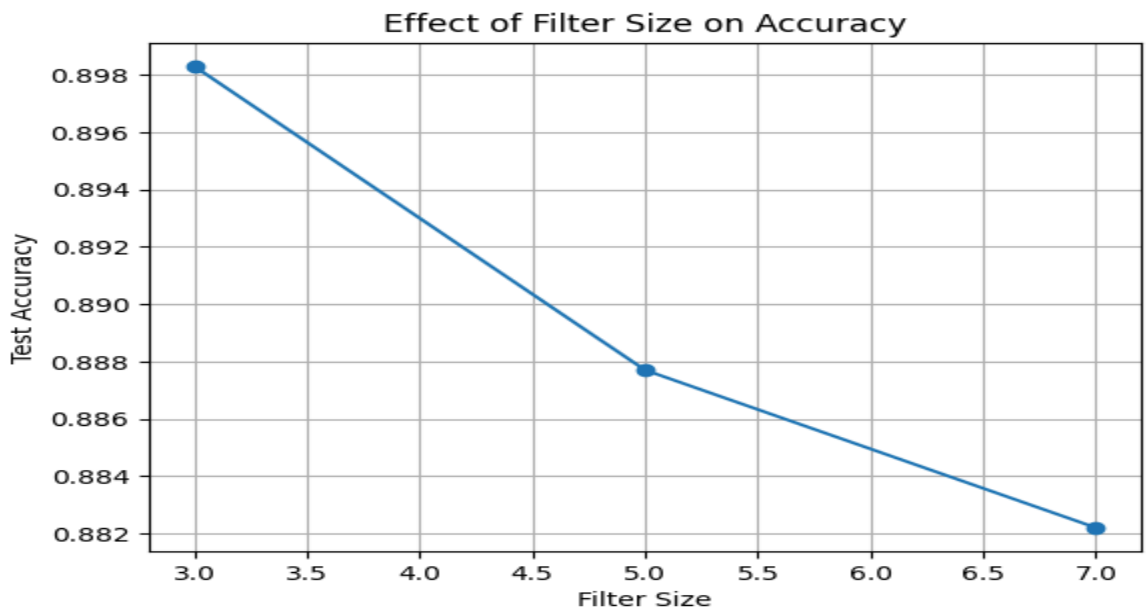


EXPERIMENT 1 : Effect of Filter Size

```
filter_sizes = [3,5,7]
filter_accuracy = []
for f in filter_sizes:
    print("\nTraining with filter size:",f)
    model = create_model(filter_size=f)
    model.fit(
        x_train,
        y_train,
        epochs=5,
        batch_size=64,
        validation_split=0.2,
        verbose=1
    )
```

```
    loss,acc = model.evaluate(x_test,y_test)

    filter_accuracy.append(acc)

plt.figure()

plt.plot(filter_sizes,filter_accuracy,marker='o')

plt.title("Effect of Filter Size on Accuracy")
plt.xlabel("Filter Size")
plt.ylabel("Test Accuracy")
plt.grid(True)
plt.show()
```

Output :-

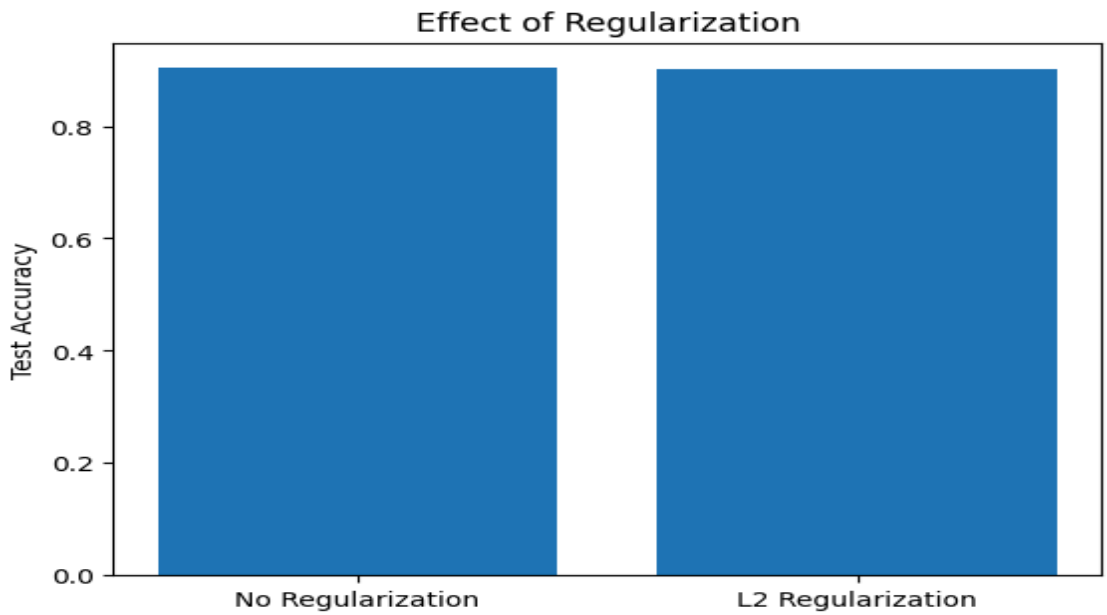


EXPERIMENT 2 : Effect of Regularization

```
reg_types = [False,True]
reg_accuracy = []
for r in reg_types:

    print("\nTraining with Regularization:",r)
    model = create_model(regularization=r)
    model.fit(
        x_train,
        y_train,
        epochs=5,
```

```
        batch_size=64,
        validation_split=0.2,
        verbose=1
    )
    loss,acc = model.evaluate(x_test,y_test)
    reg_accuracy.append(acc)

plt.figure()
labels = ["No Regularization","L2 Regularization"]
plt.bar(labels,reg_accuracy)
plt.title("Effect of Regularization")
plt.ylabel("Test Accuracy")
plt.show()
```

Output :-

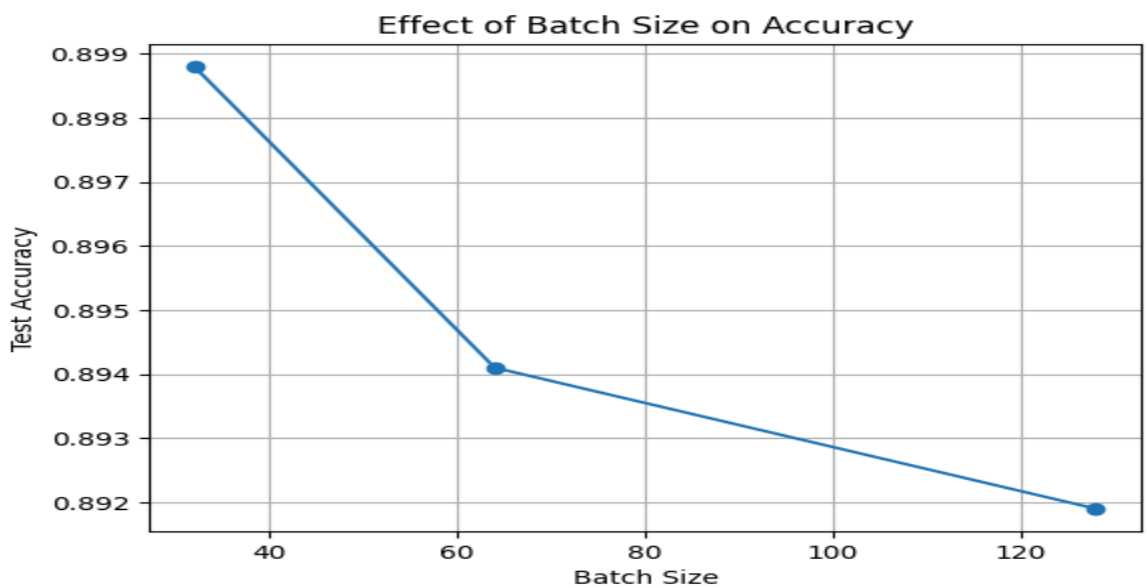


EXPERIMENT 3 : Effect of Batch Size

```
batch_sizes = [32,64,128]
batch_accuracy = []
for b in batch_sizes:
    print("\nTraining with batch size:",b)
    model = create_model()
    model.fit(
        x_train,
        y_train,
```

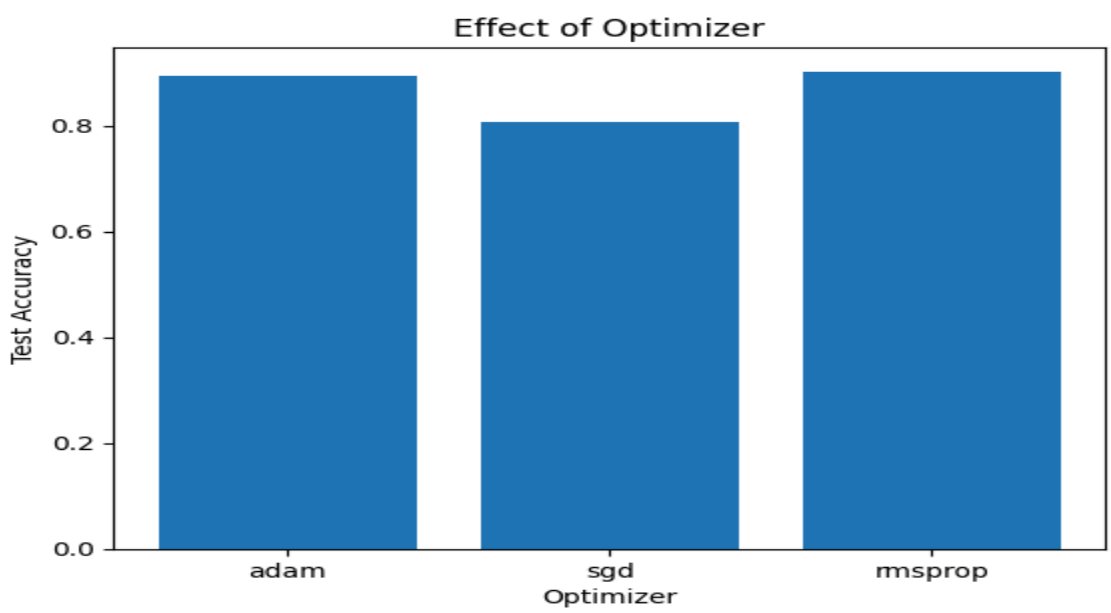
```
        epochs=5,
        batch_size=b,
        validation_split=0.2,
        verbose=1 )
    loss,acc = model.evaluate(x_test,y_test)
    batch_accuracy.append(acc)

plt.figure()
plt.plot(batch_sizes,batch_accuracy,marker='o')
plt.title("Effect of Batch Size on Accuracy")
plt.xlabel("Batch Size")
plt.ylabel("Test Accuracy")
plt.grid(True)
plt.show()
```

Output :-



EXPERIMENT 4 : Effect of Optimizer

```
optimizers = ['adam','sgd','rmsprop']
opt_accuracy = []
for opt in optimizers:
    print("\nTraining with optimizer:",opt)
    model = create_model(optimizer=opt)
    model.fit(
        x_train,
        y_train,
```

```
        epochs=5,
        batch_size=64,
        validation_split=0.2,
        verbose=1
    )
    loss,acc = model.evaluate(x_test,y_test)
    opt_accuracy.append(acc)

plt.figure()
plt.bar(optimizers,opt_accuracy)
plt.title("Effect of Optimizer")
plt.xlabel("Optimizer")
plt.ylabel("Test Accuracy")
plt.show()
```

Output :-